

Exercise 3 : Simulate error correction mechanism

Program:

```
def xor(dividend,divisor):
    result=""
    for i in range(1,len(divisor)):
        if(dividend[i]==divisor[i]):
            result+="0"
        else:
            result+="1"
    return result

def mod2div(dividend,divisor):
    pick=len(divisor)
    tmp=dividend[0:pick]
    while pick<len(dividend):
        if tmp[0]=='1':
            tmp=xor(tmp,divisor)+dividend[pick]
        else:
            tmp=xor(tmp,'0'*len(divisor))+dividend[pick]
        pick+=1
    if tmp[0]=='1':
        tmp=xor(tmp,divisor)
    else:
        tmp=xor(tmp,'0'*len(divisor))
    return tmp

data=input("Enter Data(Dividend):")
divisor=input("Enter Divisor (Generator Polynomial):")
appended_data=data+"0"*(len(divisor)-1)
crc=mod2div(appended_data,divisor)
```

```

codeword=data+crc
print("\n CRC Parity Bits",crc)
print("Transmitted Codeword :",codeword)
received = input("\nEnter Received Codeword: ")
if len(received) != len(codeword):
    print("\nInvalid input!")
    print("Please enter the complete received codeword of length", len(codeword))
else:
    remainder = mod2div(received, divisor)

    if '1' in remainder:
        print("\nResult : Error Detected (Single/Multiple Bit Error)")
    else:
        print("\nResult : No Error Detected")

```

Output:

```

Python 3.13.14 (tags/v3.13.14:fd1/99/, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:\Users\Sujitha\Documents\ex1_cn.py =====
Enter Data(Dividend):1001
Enter Divisor (Generator Polynomial):1011

CRC Parity Bits 110
Transmitted Codeword : 1001110

Enter Received Codeword: 1001110

Result : No Error Detected

===== RESTART: C:\Users\Sujitha\Documents\ex1_cn.py =====
Enter Data(Dividend):10111010
Enter Divisor (Generator Polynomial):10000010

CRC Parity Bits 1110000
Transmitted Codeword : 101110101110000

Enter Received Codeword: 101110101110000

Result : No Error Detected
|

```

Exercise 3 : Simulate error correction mechanism

Program:

```
parity=input("Enter parity type (even/odd):").strip().lower()
data=input("Enter the binary data:").strip()
count_ones=data.count('1')
print("\n Parity generation:")
if parity=="even":
    if count_ones%2==0:
        parity_bit="0"
    else:
        parity_bit="1"
    data=data+parity_bit
    print("Selected Parity:Even Parity")
    print("Data with parity bit:",data)
elif parity=="odd":
    if count_ones%2==1:
        parity_bit="0"
    else:
        parity_bit="1"
    data=data+parity_bit
    print("Selected Parity:Odd Parity")
    print("Data with parity bit:",data)
else:
    print("Invalid parity type! Enter either 'even' or 'odd!'")
```

Output:

```
===== RESTART: C:\Users\Sujitha\Documents\ex2_cn.py =====  
Enter parity type (even/odd):odd  
Enter the binary data:101  
  
Parity generation:  
Selected Parity:Odd Parity  
Data with parity bit: 1011  
  
===== RESTART: C:\Users\Sujitha\Documents\ex2_cn.py =====  
Enter parity type (even/odd):even  
Enter the binary data:101  
  
Parity generation:  
Selected Parity:Even Parity  
Data with parity bit: 1010
```